

TLMGame - Jogo de treinamento de raciocínio lógico e matemático para todas as idades

Projeto Final - Unidade 7



Autor:

Vinícius Esperança Mantovani

Escopo do projeto:

Apresentação do Projeto:

O projeto desenvolvido se trata de um jogo de raciocínio lógico desenvolvido em uma Raspberry Pi Pico W sobre a estrutura de uma Bitdoglab. Tal jogo é composto por dois modos, o de lógica e o de matemática. No primeiro modo, o jogador é convidado a completar o quarto “quadro” dos quatro apresentados na matriz de LEDs da Bitdoglab, como em um teste de QI com sequências de figuras que seguem determinada lógica. Para tanto, o jogador deve observar a transformação ocorrida do primeiro para o segundo quadro (do esquerdo para o direito de cima), entender a lógica da transformação e reproduzi-la na sequência de baixo, compondo o da direita a partir do da esquerda (a única ligação dos quadros de baixo para os de cima é a lógica de transformação do primeiro para o segundo). Já no outro modo, o de matemática, o jogador tem de observar as contas de um único operador que aparecem na matriz de LEDs e, na sequência, responder o resultado da conta da fase atual. Desse modo, no primeiro jogo o usuário recebe a oportunidade de treinar seu raciocínio lógico e, no segundo, tem a chance de praticar seu raciocínio matemático.

Como adendo, destaca-se a referência de número 11, a qual contém o *link* para o repositório do projeto no Github.

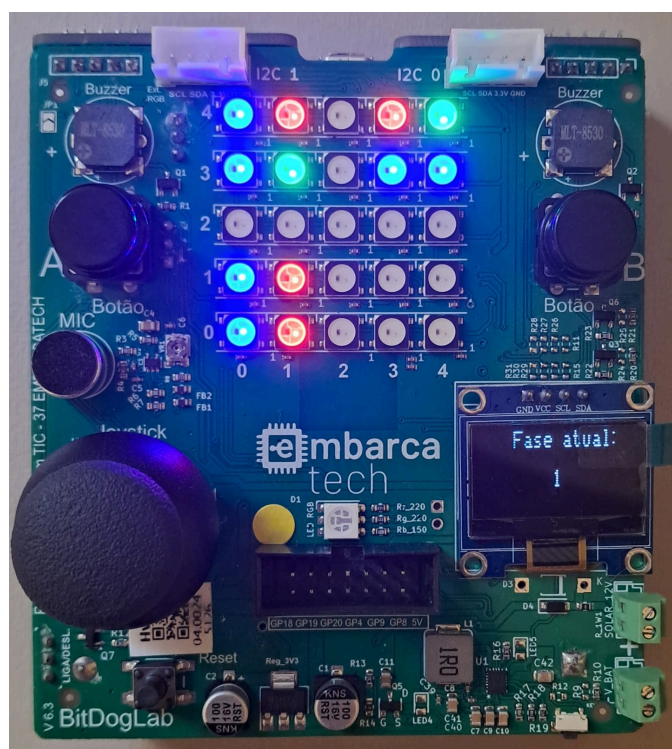


Figura 1: Fase inicial do jogo de lógica (TLogicM, fases geradas aleatoriamente).

Título do Projeto:

O jogo foi nomeado “TLMGame” por conta de sua natureza, significando: “The Logic and Math Game”. Em português: “O jogo de Lógica e Matemática”.

Objetivos do Projeto:

Os objetivos do projeto, embora bem definidos, são razoavelmente diversos. Isso porque, ainda que o TLMGame vise diretamente a treinar o raciocínio lógico-matemático dos usuários, ele é feito para atender às mais diversas faixas etárias. Tem como objetivo ajudar jovens e crianças a se

desenvolverem na escola, melhorando sua noção de lógica e permitindo seu desenvolvimento no raciocínio matemático, mas também, de maneira um pouco distante do já apresentado, o jogo busca ajudar as mais diversas pessoas a exercitarem seus cérebros de uma forma divertida e eficiente, principalmente no que tange ao raciocínio lógico e matemático, permitindo-as ter maiores noções matemáticas para lidar com os mais diversos problemas do dia-a-dia e, ainda possivelmente, com as questões financeiras que as seguirão por toda a vida.

Principais Requisitos:

O projeto partiu de requisitos específicos determinados anteriormente a sua execução e, alguns, pensados até mesmo durante sua execução como complementos. Esses requisitos foram pensados já tendo em vista o que há de possibilidades na placa Bitdoglab, sendo eles:

1. Deve estimular o usuário a pensar racionalmente;
2. Deve conter dois jogos, ambos compostos por fases geradas aleatoriamente sob critérios a determinar;
3. Os jogos devem conter um tutorial prévio antes de iniciarem efetivamente, explicando como o jogador pode executar cada ação e como o jogo funciona:
 - 3.1. Deve ser possível pular o tutorial caso o jogador deseje.
4. Os jogos devem ter sons para acertos, derrotas (erros) e mudanças de fase.
5. Deve haver a opção de jogar com ou sem som;
6. Deve conter um jogo de lógica, sobre o qual se afirma:
 - 6.1. É baseado em duas sequências de dois “quadros” (unidades visuais do jogo) empilhadas uma em cima da outra:
 - 6.1.1. Cada um dos quatro quadros deve possuir quatro pontos coloridos (LEDs) que possam variar entre quatro cores: “branco” (LED apagado), “azul”, “verde” e “vermelho”;
 - 6.1.2. Os quadros da esquerda, das sequências de cima e de baixo, devem ser gerados aleatoriamente, ou seja, cada um dos LEDs que os compõem poderia assumir qualquer uma dentre as quatro cores. No entanto, isso deve ser feito de forma a evitar que a sequência de cima tenha uma lógica confusa entre duas possíveis transformações (entenda-se a que se refere ao citar “lógicas de transformações” nos próximos requisitos);
 - 6.1.3. Na sequência de cima, deve ser gerado o quadro da direita (pelo programa), a partir de alguma lógica de transformação do quadro da esquerda, já gerado aleatoriamente;
 - 6.1.4. O segundo quadro de baixo, deve ser deixado em branco para que o jogador o preencha;
 - 6.1.5. O jogador deve ter que preencher o segundo quadro de baixo partindo da lógica de transformação usada na geração do segundo de cima para transformar o primeiro quadro de baixo. Por exemplo, caso o segundo de cima seja gerado por uma rotação de 180 graus do primeiro, o segundo de baixo deve ser gerado pelo jogador por uma rotação de 180 graus do primeiro de baixo.
 - 6.1.6. O jogador deve ter a possibilidade de manipular as cores de todos os LEDs do último quadro, mas não dos demais;
 - 6.1.7. Caso o jogador acerte as cores esperadas para o último quadro, uma nova fase deve ser gerada para ele, do mesmo modo, com cores aleatórias a não ser das

- imposições anti-problemas e, mais uma vez, o jogador deve tentar acertar o último quadro;
- 6.1.8. Caso o jogador erre, deve receber uma mensagem de derrota e o programa deve voltar ao início para que o usuário, seja ele o mesmo ou não, possa escolher novamente o jogo que deseja jogar.
 - 6.2. Quanto à lógica de transformação citada acima:
 - 6.2.1. Deve haver a possibilidade de rotação (90 graus no sentido anti-horário ou no horário ou, 180 graus);
 - 6.2.2. Deve haver a possibilidade de mudança de cores de componentes do primeiro quadro para o segundo quadro de uma sequência;
 - 6.2.3. Não deve haver a mistura de duas ou mais possíveis transformações, ou seja, não deve haver, por exemplo, uma sequência de rotações de um quadro para o outro nem uma mescla de rotação e mudança de cores (para evitar lógicas que possam causar confusão com outras lógicas, ou seja, lógicas que não seriam as únicas possíveis para as transformações apresentadas);
 - 6.2.4. A lógica é escolhida aleatoriamente pelo programa entre as citadas.
 - 7. Deve conter um jogo de matemática, sobre o qual se afirma:
 - 7.1. A cada fase deve ser gerada uma conta matemática de soma, subtração, multiplicação ou divisão;
 - 7.1.1. A conta gerada deve ser apresentada para o jogador;
 - 7.1.2. A dificuldade das contas deve aumentar progressivamente com base na fase em que o jogador se encontra: quanto maior a fase mais difícil a conta.
 - 7.2. Deve haver a possibilidade do jogador escrever sua resposta e enviá-la ao programa para verificação. Assim:
 - 7.2.1. Caso acerte, uma nova conta é gerada e o jogador deve, novamente, chegar ao seu resultado para prosseguir;
 - 7.2.2. Caso o usuário erre, deve receber uma mensagem de derrota com o valor correto. Após isso o programa deve voltar ao início para que o usuário, seja ele o mesmo ou não, possa escolher novamente o jogo que deseja jogar.

Descrição do Funcionamento:

O projeto desenvolvido inicia com a apresentação de uma mensagem de “bem-vindo” no display OLED existente na Bitdoglab e, segue com mensagens questionando o usuário. Este por sua vez, usa botões de pull-up nomeados A e B da placa para responder às perguntas feitas pelo programa e, desse modo, determinar o que acontecerá em seguida.

Inicialmente, o programa pergunta ao usuário se ele deseja usá-lo com som ou sem (A - com som, B - sem som). Em seguida, o usuário é questionado sobre qual jogo deseja jogar, podendo escolher entre o “TLogicM” (A) ou o “TLMath” (B). Tendo escolhido o jogo, o usuário recebe uma última pergunta, se deseja ou não ler o tutorial do jogo (A - sim, B - não). Em caso afirmativo, são apresentadas mensagens de tutorial, ainda no display, e o jogador pode avançar de uma mensagem para a próxima pressionando A. Após essas etapas, a placa apresenta ao usuário a primeira fase do jogo escolhido, iniciando-o.

No jogo de Lógica, o jogador recebe, a cada fase, três quadros de quatro LEDs preenchidos: os da esquerda com quatro cores aleatórias seguindo algumas restrições e o de cima da direita originado de uma transformação do primeiro. Tendo isso, o usuário deve usar o joystick para se movimentar entre os LEDs do quadro da direita de baixo, o botão A para escolher entre as quatro cores citadas nos requisitos e o botão B para confirmar a escolha de cor para um LED. Assim, definidas as cores de todos os LEDs, o usuário deve pressionar o joystick para confirmar sua escolha

final. Com isso, tendo ele acertado, o jogo continua para uma nova fase como a última. No entanto, caso o jogador erre, o programa reinicia e o usuário passa por todo o trâmite explicado.

Já no jogo de Matemática, o jogador recebe uma conta a cada fase, de maneira que há apenas uma operação e cada caractere (incluindo o operador) é apresentado durante 500 milissegundos na matriz de LEDs (com intervalos de um para o próximo também de 500 ms). Dada a fase, o programa apresenta um conjunto de zeros no display, sob um dos quais se encontra um traço, que indica que esse é o algarismo atualmente modificável pelo usuário. Nesse sentido, para alterar tal algarismo, o jogador deve movimentar o joystick para cima para aumentá-lo ou para baixo para diminuí-lo. Seguindo, caso seja necessário alterar outros algarismos, o botão B permite navegar para algarismos mais significativos e o A para aqueles menos significativos. Finalmente, quando o usuário avançar para a direita com o A para além do algarismo menos significativo, o resultado dele é registrado e é analisado pelo programa. Caso esteja correto, o jogador recebe mensagem de vitória, caso contrário, recebe mensagem de derrota apresentando o resultado correto e, conforme citado no jogo anteriormente explicado, o programa reinicia.

Em suma, conforme já apresentado, o projeto tem como funcionalidades dois jogos para treinamento de raciocínio lógico e matemático.

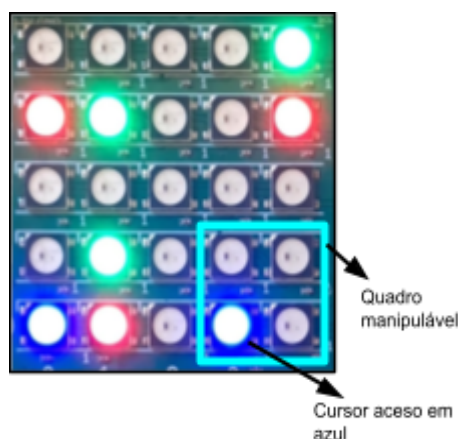


Figura 2: Demo TLogicM.

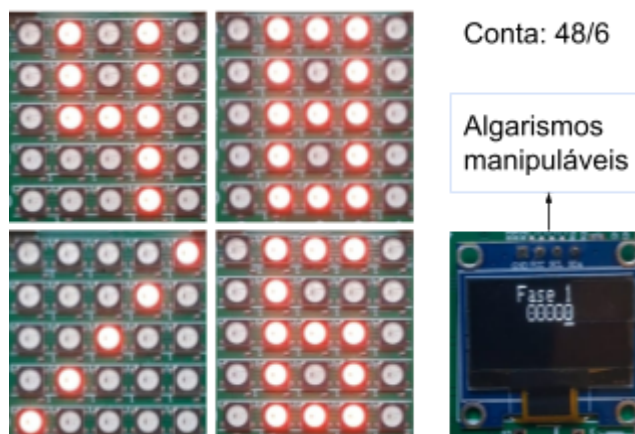


Figura 3: Demo TLMath.

Justificativa:

Conforme se pode notar ao analisar os dados registrados pelo “TIMSS” (International Results in Mathematics and Science), há um problema de defasagem na aprendizagem de matemática no Brasil a níveis de ensinos fundamental. Isso porque, tais dados apontam que em 2023 entre os 58 países analisados o Brasil fica em 55º lugar no ranking no que tange a alunos do 4º ano e em 43º entre os 44 países em relação ao 8º ano. Nesse sentido, o projeto desenvolvido encontra seu motivo de existir no fato de conciliar uma ferramenta de treinamento de uma habilidade tão necessária para as pessoas e tão deficitária no país, a matemática, ao entretenimento super valorizado por jovens, os jogos. Ainda nesse contexto, expandindo o problema citado que se refere a um contexto escolar, pode-se notar que, justamente por ser o conhecimento escolar o fundamento para o conhecimento das pessoas no resto de suas vidas e a bússola que as guia nos desafios diários, o problema não fica apenas na escola, mas segue os cidadãos no seu cotidiano. Assim, o jogo se mostra uma forma divertida para todas as idades de se treinar as habilidades destacadas, permitindo que as pessoas consigam ter mais facilidade e praticidade com tarefas que exijam noções matemáticas e lógicas.

Originalidade:

Após busca cautelosa por projetos semelhantes ao aqui exposto, foram encontrados alguns poucos que se assemelham principalmente ao jogo de matemática. O primeiro desses jogos (link no repositório Github de projetos correlatos, nas referências) é um jogo de quiz de matemática, semelhante ao exposto aqui. No entanto, é desenvolvido em Python e tem suas saídas e entradas diretamente no terminal, sendo, portanto, diferente do TLMGame por não possuir nenhum dos periféricos aqui usados, pela simplicidade do jogo quando comparado ao TLMGame e, também neste sentido, pela falta de lógica de progressão de dificuldade, que existe no TLMGame e não no jogo citado. Para além desse jogo, porém, há outro que menos se relaciona com o TLMGame na parte de matemática, uma vez que é feito diretamente para ser rodado no computador, não em uma Pi Pico. Ainda assim, há um último projeto razoavelmente semelhante ao presente no que se refere ao jogo de matemática, o projeto apresentado no segundo link do repositório de correlatos. Em tal projeto é desenvolvido um jogo de matemática parecido com o TLMath, uma vez que é desenvolvido com um display e leds. Porém, também é um projeto em Python e se difere muito na questão de progressão do que se pode perceber por seu código, além de não apresentar as questões da matriz de LED e outros periféricos (joystick, botões e buzzers). Ademais, quanto ao TLogicM, apesar da busca aprofundada, não foi encontrado nenhum projeto que se assemelhasse razoavelmente a ele, provavelmente por sua especificidade muito acentuada.

Em suma, após pesquisa acalorada por projetos correlatos, o TLMGame se apresenta como um projeto original e de funcionalidades diferenciadas dos encontrados em buscas na internet.

Hardware:

Diagrama em Blocos:

A seguir, encontra-se um diagrama de blocos sintetizando cada componente de hardware do sistema desenvolvido e as integrações entre eles:

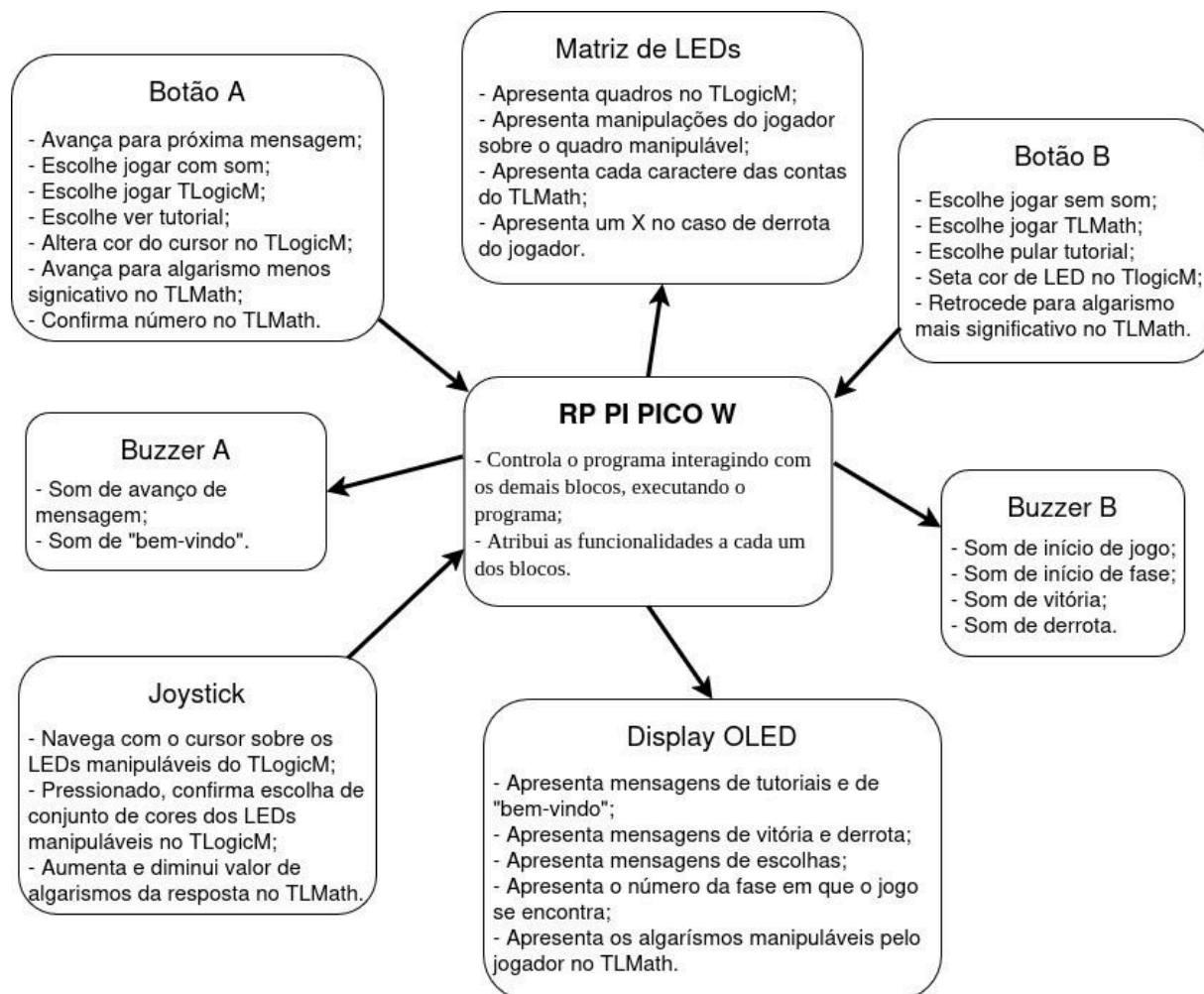


Figura 4: diagrama de blocos do hardware envolvido no TLMGame.

Função de cada bloco:

Dado o diagrama de blocos, pode-se descrever a função de cada bloco nele presente. Para tanto, demonstra-se a seguir a função de cada um deles em três diferentes etapas do projeto:

1. Raspberry Pi Pico W - Responsável por interagir com cada um dos demais blocos, controlando o programa para a execução de cada passo. Atribui as funcionalidades a cada um dos blocos.
2. Momento pré-jogo:
 - 2.1. Botão A - Permite que o usuário avance de mensagem a mensagem apresentada no display. Além disso, nos momentos em que é indagado pelo programa por meio do display a respeito de cada possibilidade, o usuário pode pressionar A para escolher jogar com som, jogar TLogicM e/ou ver o tutorial (conjunto de mensagens explicativas dentre as quais se avança, também com A);

- 2.2. Botão B - Permite ao usuário escolher jogar sem som, jogar TLMath e/ou pular o tutorial;
 - 2.3. Buzzer A - Toca sons de avanço de uma mensagem a outra e som de “welcoming” no início do programa, caso o modo selecionado não seja mudo. Neste caso, mudo, apenas toca o som de “welcoming” e é silenciado logo após tocá-lo.
 - 2.4. Buzzer B - Após todas as mensagens e escolhas do usuário, toca som de início de jogo (caso não esteja no modo mudo);
 - 2.5. Display - Apresenta mensagens de “welcoming” no início do programa, de escolha sobre som, jogo e pular tutorial e, mensagens do tutorial (caso o usuário opte por não pulá-las);
 - 2.6. Joystick e Matriz de LEDs - Não atuam nesse momento.
3. Durante o TLogicMath:
 - 3.1. Botão A - Permite que o jogador altere a cor de seu “cursor” (indicador do LED da matriz que está manipulando no momento);
 - 3.2. Botão B - Permite que o jogador defina a cor do LED sobre o qual seu cursor pousa no momento como sendo a cor de seu “cursor” (exemplo: cursor sobre LED 0 com cor azul. Ao pressionar B, por mais que ele opte por tirar o cursor de cima de tal LED, ele permanecerá azul).
 - 3.3. Buzzer B - Sons de início de fase, de vitória e de derrota;
 - 3.4. Display - Apresenta o número da fase em que o jogador se encontra, além de mensagens de vitória e derrota;
 - 3.5. Joystick - Permite que o jogador navegue com seu cursor sobre os quatro LEDs que pode manipular, os LEDs do quadro direito de baixo. Ademais, quando pressionado, confirma a escolha de conjunto de cores dos LEDs definitiva para que seja conferida sua validade e o programa prossiga para uma vitória de fase ou derrota;
 - 3.6. Matriz de LEDs - apresenta os quatro quadros de quatro LEDs cada, duas sequências horizontais, uma sobre a outra, de modo a apresentar cada uma das fases ao jogador e, também, as mudanças de cor feitas por ele no quadro manipulável. Finalmente, em caso de derrota, apresenta um “X”.
 - 3.7. Buzzer A - Não atua nesse momento.
4. Durante o TLMath:
 - 4.1. Botão A - Permite que o usuário navegue de um algarismo mais significativo para um menos significativo. Caso o algarismo em que o usuário se encontra no momento seja o menos significativo, o pressionamento de A envia o valor registrado pelo usuário para ser conferido e o programa prosseguir com vitória ou derrota na fase;
 - 4.2. Botão B - Permite que o usuário navegue de um algarismo menos significativo para um mais significativo.
 - 4.3. Buzzer B - Sons de início de fase, de vitória e de derrota;
 - 4.4. Display - Apresenta o número da fase em que o jogador se encontra e os algarismos que pode manipular para escrever seu resultado (5 algarismos ao todo, iniciando em 0 a cada fase). Mostra também, mensagens de vitória da fase e de derrota acompanhada da resposta esperada (resultado correto da conta a ser feita);
 - 4.5. Joystick - Permite que o usuário, empurrando-o para cima, aumente um algarismo até 9 (chegando em 9, caso continue a empurrá-lo, o usuário verá o algarismo voltar a 0 e continuar subindo a partir daí). Do mesmo modo, caso o empurre para baixo, o usuário pode reduzir o valor do algarismo (de modo semelhante à subida, vai de 9 a 0

e, caso passe de 0, volta para 9 e continua descendo a partir daí). Ainda assim, ao contrário do TLogicM, neste jogo o pressionamento do joystick não exerce função alguma;

- 4.6. Matriz de LEDs - Apresenta caractere por caractere presentes na conta para o usuário, de modo que, por exemplo, em uma conta de $35+7$, a matriz mostraria primeiro o 3, depois o 5, depois o “+” e finalmente o 7, com um intervalo de 500 ms entre cada caractere e uma duração também de 500 ms para cada um deles na matriz. Finalmente, em caso de derrota, apresenta um “X”.
- 4.7. Buzzer A - Não atua nesse momento.

Configuração de cada bloco:

Como modo de facilitar o entendimento, novamente, separa-se cada bloco em itens para atribuir-lhes suas respectivas configurações:

1. Botões A e B - Conectados respectivamente aos GPIO5 e GPIO6, ambos configurados com resistores de pull-up internos;
2. Buzzers passivos (MLT-8530) A e B - Conectados respectivamente aos GPIO21 e GPIO10, estando tais GPIOs configurados como saídas dos PWM2 B e PWM5 A;
3. Display OLED (ssd1306) - Conectado canal I2C1 da placa, com SDA no GPIO14 e SCL no GPIO15. Configurado para trocar dados por I2C1 com a placa;
4. Matriz de LEDs (WS2812B) 5x5 - Conectada ao GPIO7, que opera com uma máquina PIO para troca de dados entre placa e matriz;
5. Joystick analógico (KY023) - VRY e VRX, respectivamente medidas no eixo y e x do joystick, conectam-se respectivamente aos GPIO26 e GPIO27. Ambas as saídas do joystick são lidas por ADCs presentes, respectivamente, ADC0 no GPIO26 e ADC1 no GPIO27. Já SW (o botão contido no joystick), atua como os demais botões, estando conectado ao GPIO22 e ao GND, sendo esse GPIO configurado com resistores pull-up internos;

Especificações:

Para explicar como as especificações técnicas do hardware atendem aos requisitos estabelecidos, pode-se também organizar itens, conforme se segue:

1. Mensagens - Todas as mensagens, incluindo tutoriais, questionamentos, mensagens de derrota, vitória e todas as demais informações textuais podem ser apresentadas no display, incluindo ainda informações como a fase atual do jogador e o valor dos algarismos manipuláveis por ele no caso do TLMath;
2. Sons - Todos os sons emitidos durante a execução do programa podem ser tocados por meio dos dois buzzers presentes no sistema desenvolvido;
3. Interface visual principal dos jogos - Para ambos os jogos, a matriz de LEDs é usada para apresentar a interface visual mais significativa. No TLMath, a matriz apresenta os caracteres da conta a ser resolvida, conforme explicado anteriormente. Já no caso do TLogicM, ela apresenta os quatro quadros do jogo e é nela que o jogador vê as mudanças de cor do quadro manipulável acontecerem. Isso é possível, pois a matriz contém LEDs RGB, possibilitando a existência de quatro diferentes cores requisitadas no jogo;
4. Movimentos nos jogos - Conforme já explicado a respeito dos movimentos do jogador durante os jogos, no TLogicM, o joystick funciona como forma de o jogador mover o cursor entre os quatro LEDs do quadro manipulável (quadro da direita de baixo) e, seu pressionamento funciona como confirmação de sua escolha para o conjunto de cores. Além

disso, para manipular efetivamente um LED da matriz, o jogador pode escolher a cor usando o botão A, avançando da cor branca à vermelha (branca -> azul -> verde -> vermelho -> branca...). Finalmente, pode-se usar o botão B para *setar* a cor do LED sobre o qual pousa o cursor do jogador com a cor do cursor. Ademais, no caso do TLMath, o jogador pode escrever sua resposta da forma como já explicado anteriormente: usando o botão A para navegar para algarismos menos significativos e o B para mais significativo e, o joystick para alterar o valor de um algarismo.

Lista de Materiais:

Os materiais usados foram:

- 1 - Raspberry Pi Pico W - placa desenvolvida sobre o microcontrolador RP2040, com 2MB de memória flash, interface wireless com 2,4 GHz, 30 GPIO, canais I2C, SPI, UART, ADC, PWM e outras *features* ;
- 2 - Buzzers passivos MLT-8530 - buzzers passivos eletromagnéticos;
- 2 - botões push buttons - Push buttons simples;
- 25 - LEDs RGB WS2812B - LEDs de controle inteligente ligados em série para uso de poucos pinos para comunicação. Apresentam velocidade de envio de dados igual a 800 Kbps, tendo circuito de controles e chips RGB somando 5050 componentes;
- 1 - Display OLED - Display OLED com chip ssd1306, resolução e buffer SRAM de 64x128 (linhas x colunas) com 0,96 polegadas e comunicação I2C;
- Joystick analógico KY023 - Joystick com saídas analógicas para os eixos x e y e saída digital para o eixo z (como um push button).

Descrição da pinagem usada:

Para execução do projeto, foram usados os GPIOs 5 e 6 configurados com resistores de pull-up internos para ligação dos botões A e B, respectivamente, o GPIO 7 para a matriz de LEDs com configuração usando máquina PIO, os GPIOs 10 e 21 como saídas de canais PWM para ligação com os buzzers B e A, respectivamente e, os GPIOs 14 e 15 para o Display OLED usando comunicação I2C (14 - SDA, 15 - SCL). Além disso, tem-se o Joystick com botão (eixo z) ligado ao GPIO 22, com mesma configuração que para os botões A e B e, leituras analógicas VRY e VRX, dos eixos x e y, respectivamente, ligados aos GPIOs 26 e 27, respectivamente, conversores analógico digital 0 e 1. Finalmente, vale destacar que todos os periféricos estão ligados a pinos de *ground* da placa e, com exceção dos botões e dos buzzers, estão todos conectados a saídas de alimentação da placa.

Abaixo encontra-se uma tabela apresentando as integrações da placa com os periféricos, suprimindo-se pinos de VOUT e GND:

GPIO	CANAL/INTERAÇÃO	LINHA	PERIFÉRICO
5	-	-	Botão A
6	-	-	Botão B
7	PIO	-	Matriz de LEDs
10	PWM5 A	-	Buzzer B
14	I2C 1	SDA	Display OLED

15	I2C 1	SCL	Display OLED
21	PWM2 B	-	Buzzer A
22	-	-	SW do Joystick
26	ADC 0	-	VRX do Joystick
27	ADC 1	-	VRX do Joystick

Circuito Completo do Hardware:

Como forma de ilustrar o que foi explicado até aqui a respeito do hardware usado no projeto presente, apresenta-se a seguir uma figura contendo todo o circuito de hardware:

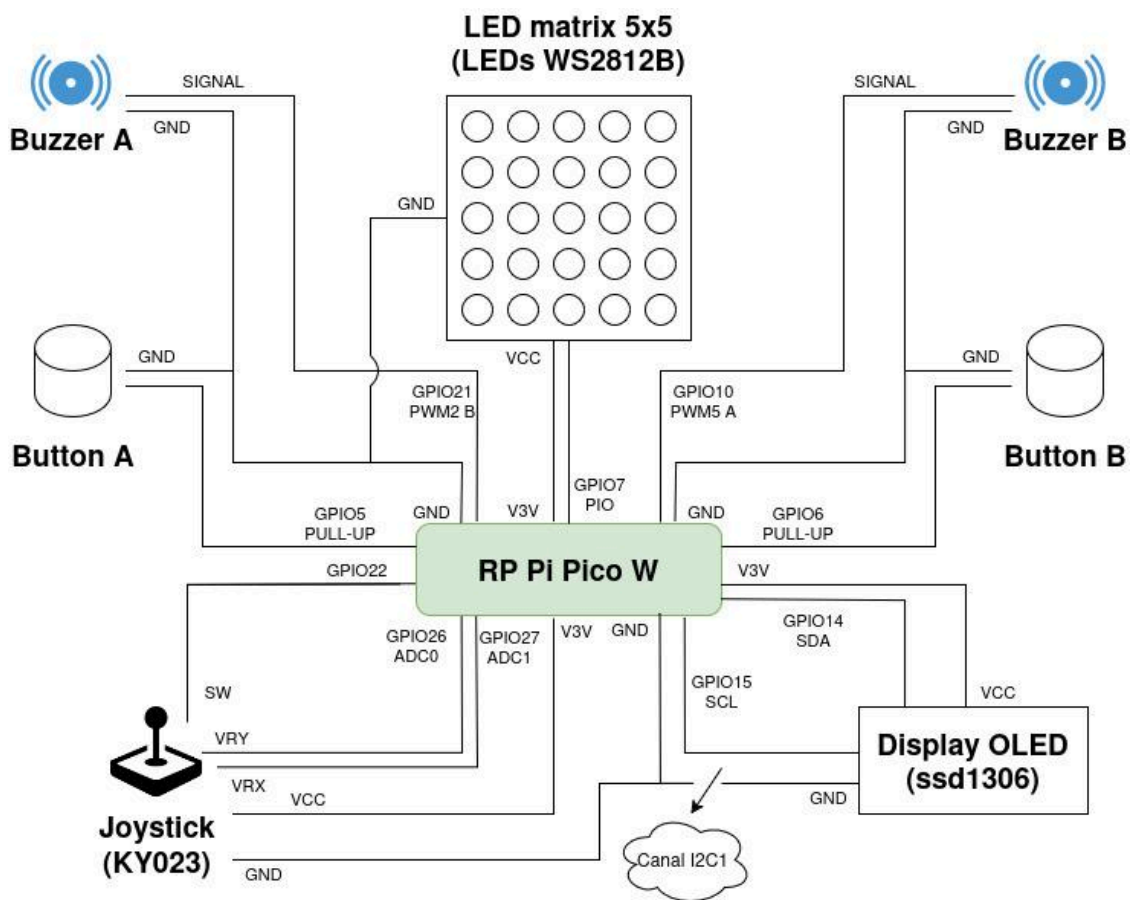


Figura 5: Desenho do Circuito do Projeto.

Software:

Blocos Funcionais:

A seguir, apresentam-se um diagrama de blocos com cada camada do software desenvolvido para o projeto e uma descrição curta do que fazem e/ou contêm:

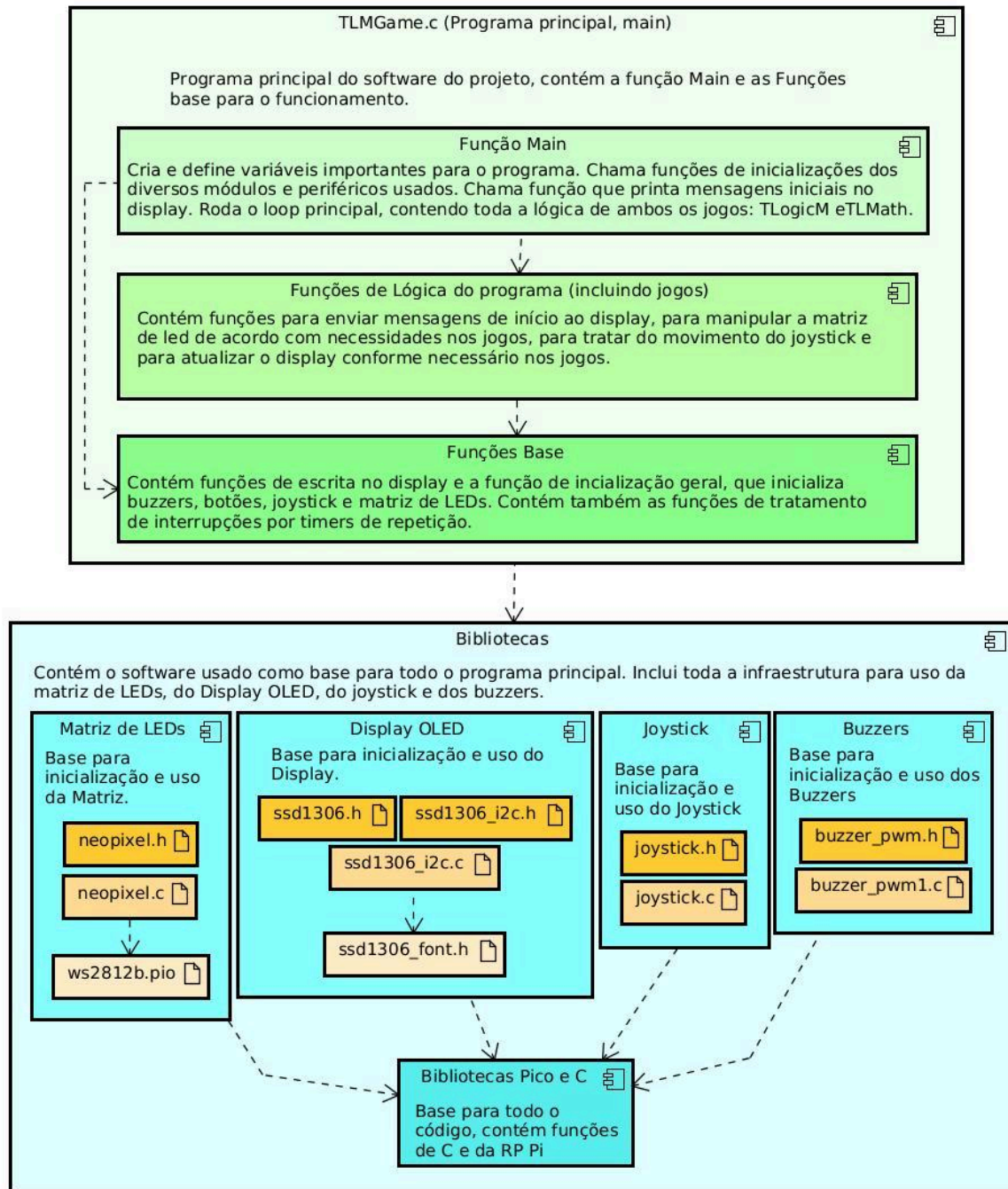


Figura 6: Diagrama de blocos das camadas de software do programa. A seta indica dependência, de forma que aquele que depende aponta para aquele do qual depende (A depende de B = $A \rightarrow B$).

Descrição das Funcionalidades:

Para descrever as funcionalidades de cada camada de software, vale iniciar pelas bibliotecas, base de todo o projeto. Para tanto, vale ainda ressaltar que as bibliotecas chamadas no diagrama da figura 6 de “Matriz de LEDs”, “Display OLED”, “Joystick” e “Buzzers” foram feitas a partir de

códigos prontos de exemplos em C para uso de componentes da Bitdoglab (números 13 a 16 nas referências). Isso se deu, de modo que alguns desses códigos foram alterados (alterações descritas em comentários nos códigos), como no caso do display, em que os códigos foram alterados para que fossem adicionados novos caracteres na fonte de escrita nele. Além disso, como os códigos encontrados nos números citados das referências não apresentam arquivos de cabeçalho, esses arquivos foram criados para que fosse possível usar os códigos como bibliotecas (os únicos cabeçalhos já existentes eram os `ssd1306.h`, o `ssd1306_i2c.h` e o `ssd1306_font.h`).

Nesse contexto, a biblioteca responsável pelo controle da matriz de LEDs possui um arquivo de configuração e definição da inicialização do programa PIO (`ws2812.pio`, convertido em cabeçalho por comando no `CMakeLists.txt`), um arquivo cabeçalho criado, responsável pela declaração de funções e de definições (“`#defines`”) usadas em outros arquivos de código. Finalmente, há o código principal (`neopixel.c`) que contém as funções de posse de máquina PIO e inicialização do programa PIO, de envio de dados para a matriz usando PIO e, por fim, as funções de limpeza da matriz e a de acendimento de LED.

Seguindo, a biblioteca referente ao display, contém três cabeçalhos já existentes: o “`ssd1306.h`”, contendo a definição de funções do “`ssd1306_i2c.c`”; o “`ssd1306_i2c.h`” que contém definições e estruturas usadas no funcionamento das funções em “`ssd1306_i2c.c`” e; o “`ssd1306_font.h`” que contém a estrutura definidora da fonte de escrita no display. Ademais, cabe ressaltar que o código principal dessa biblioteca, “`ssd1306_i2c.c`”, contém várias funções responsáveis por inicializar, controlar e enviar dados ao display.

Ainda no diagrama da figura 6, tem-se mais duas bibliotecas, a do joystick e a dos buzzers, sendo que, a primeira contém um código “.c” (`joystick.c`) e um “.h” (`joystick.h`). O segundo foi criado do zero para ser possível usar o “`joystick.c`” como biblioteca para o programa principal e, ele define as funções presentes em “`joystick.c`”, sendo elas a de inicialização e a de leitura de eixos do joystick. Já quanto à biblioteca dos buzzers, pode-se notar que ela também possui um “.c” e um “.h”, sendo o primeiro contenedor das funções de inicialização do buzzer e de uma que toca um tom determinado durante um tempo determinado. Para tanto, o arquivo “`buzzer_pwm.h`” possui a declaração de tais funções.

Mais acima no diagrama, tem-se a camada principal, o programa “`TLGame.c`”, que contém três subcamadas. A mais baixa, da qual as demais dependem, é a camada de funções básicas, a qual é composta por funções de *callback* ocasionadas por timers de repetição, de escrita no display (escreve um vetor de strings no display) e, uma função de inicialização, que se responsabiliza por inicializar os buzzers e botões e, chamar as funções de inicialização da matriz de led e do joystick.

Acima da subcamada explicada anteriormente, existe a subcamada de funções de lógica do programa, mais abstrata. Essa camada possui muitas funções, entre elas, uma função que cria uma fonte numérica e exibe caracteres na matriz de LEDs para o `TLMath`, uma que dispara para o display as funções de início do programa (pré-jogos) e registra escolhas do usuário quanto à existência de som, ao jogo que deseja jogar e à apresentação do tutorial. Além disso, existem funções para implementar as ações a serem tomadas no movimento do joystick para cada jogo, funções para verificar a vitória para cada jogo, uma função para reiniciar o jogo e outras mais que exercem ações sobre a lógica do programa e permitem seu funcionamento, dando infraestrutura para a função principal (`main`).

Finalmente, a camada mais superficial do software do projeto é a “`main`”. Essa camada é encarregada de ordenar o programa, chamando a função de apresentação de mensagens no início do programa e definindo o valor de variáveis usadas durante os jogos, por exemplo. No entanto, para além disso, ela define os timers repetitivos e as funções responsáveis por tratá-los (diferentes para cada jogo) e, contém ainda o “loop” infinito do programa, no qual é implementada a lógica de cada jogo, utilizando-se de chamadas de funções que participam desse processo. Entre tais funções, pode-se

citar a função de início de fase do TLogicM, que se incumbe de apagar os LEDs da última fase e acender o novo conjunto, alterar o que se exhibe no display e mais outras atividades importantes para o início de cada nova fase.

Definição das variáveis:

As principais variáveis usadas no programa desenvolvido são:

- bool game - Indica qual jogo o usuário deseja jogar (true - jogo de lógica; false - jogo de matemática). Essa variável é usada para armazenar a escolha do jogador e, durante o “loop” do programa, participar da condicional que discrimina um jogo do outro, permitindo que o programa execute o jogo correto;
- bool new_fase - Indica se o jogo deve iniciar uma nova fase ou não. Inicia como “true” para que a primeira fase seja iniciada e, existe para indicar ao programa que deve haver o início de uma nova fase no momento ou não. Recebe o valor “false” quando uma fase acaba de ser iniciada e depois, ao ocorrer uma vitória ou derrota, recebe o valor “true”;
- bool vitoria - Como insinua seu nome, é uma variável que armazena o estado pós-fase de um jogo, indicando uma vitória caso “true” ou uma derrota caso “false”. Indica ao programa se deve continuar com uma nova fase ou reiniciar. Inicia como “true” para que, com a lógica desenvolvida, seja iniciada a primeira fase de um jogo a ser iniciado;
- bool fst_in_game - booleana que indica se um jogo está sendo iniciado pela primeira vez ou pela primeira vez após a última jogada ter sido no outro jogo e, qual o jogo sendo iniciado nessas circunstâncias. A variável, é “true” quando a condição descrita é satisfeita para o TLogicM e “false” quando a condição é satisfeita para TLMath. Servem como forma de garantir que os timers de repetição sejam ativados com as funções corretas uma única vez para cada jogo em seu início. Isso porque, é nas funções de “callback” desses timers que o programa trata do uso dos botões durante os jogos, logo, caso não sejam configurados corretamente, a confusão de funções traria respostas erradas aos comandos do usuário nos jogos;
- bool com_som - Armazena a escolha do usuário sobre o programa ter som ou não (“true” - tem som, “false” - sem som);
- uint8_t posicao_atual - Armazena a posição atual do cursor no jogo de lógica;
- uint8_t cor_atual - Armazena a cor atual do cursor no jogo de lógica (cor do LED sobre o qual pousa na matriz de LEDs);
- uint8_t cor_led_0, cor_led_1, cor_led_8, cor_led_9 - Armazenam as cores de cada LED manipulável no jogo de lógica;
- uint8_t cor_led_0_verify, cor_led_1_verify, cor_led_8_verify, cor_led_9_verify - Armazenam as cores corretas para cada LED manipulável em uma fase do jogo de lógica;
- int16_t resultado_correto - Armazena o resultado correto da conta de uma fase do jogo de matemática;
- int8_t alg[5] - Armazena os 5 algarismos da resposta do jogador a uma fase do jogo de matemática;
- uint8_t which_digit - Indica o algarismo sobre o qual pousa o cursor do usuário no momento ou, em outras palavras, o algarismo alterável pelo usuário no momento, no jogo de matemática;
- uint16_t fase_atual - Armazena o número da fase em que o jogador se encontra em um jogo (inicia em zero e, ao se iniciar um jogo, recebe o valor de 1).

Fluxograma:

Abaixo, apresenta-se um fluxograma completo do software do projeto:

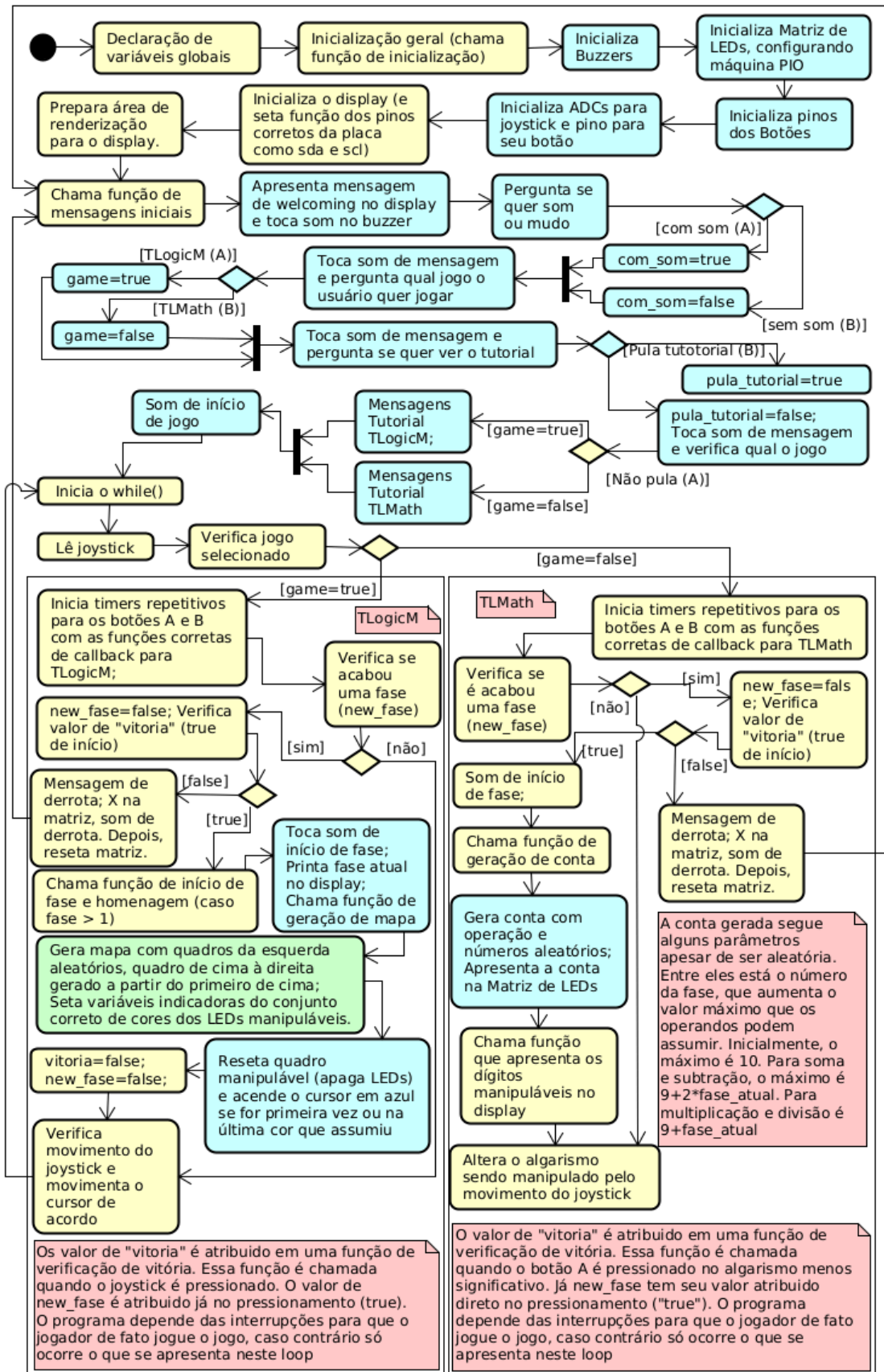


Figura 7: Fluxograma do Software do projeto.

Inicialização:

O primeiro passo dado no programa durante a inicialização é o uso da função “`stdio_init_all()`”, que se responsabiliza por inicializar as interfaces de entrada e saída padrões da PI PICO W (no caso presente, inicializa a interface I/O no USB). Então, o programa segue para a inicialização dos buzzers, feitas individualmente, por meio da mesma função “`pwm_init_buzzer()`” que se encontra no arquivo “`buzzer_pwm.c`”. Tal função usa uma chamada de “`gpio_set_function()`” para determinar que o pino ao qual se conecta o buzzer (A - GPIO21, B - GPIO10) terá função de saída PWM. Em seguida, determina-se o número do slice PWM cuja saída se apresenta no pino em questão e, declara-se uma variável para armazenar o retorno de uma chamada da função “`pwm_get_default_config()`” (retorna uma estrutura “`pwm_config`” com configurações padrão para o PWM). Dado isso, essa variável é alterada pela chamada de “`pwm_config_set_clkdiv()`”, passando a possuir um divisor de clock valendo 4 e, em seguida, é chamada a função “`pwm_init()`”, tendo como argumentos o slice, a variável de configuração e um “`true`” que indica que o PWM deve começar a rodar após a inicialização. Essa última chamada inicializa de fato o PWM e, logo na sequência, é feita uma chamada de “`pwm_set_gpio_level`” para *setar* o valor do pino como 0, garantindo que o PWM permaneça desligado inicialmente.

Em seguida, há uma chamada de “`npInit()`”, função que se inicia com a criação do programa PIO partindo do programa definido no cabeçalho “`ws2812b.pio.h`” gerado do arquivo “`ws2812b`”. Então, ainda na função, uma variável do tipo “`PIO`” tem seu valor definido como “`pio0`”, identificador da primeira instância do hardware PIO. Após isso, a função toma posse de uma máquina PIO com uma chamada “`pio_claim_unused_sm()`”, que busca por uma máquina não usada. Inicialmente, essa chamada é feita com o argumento “`false`” para o parâmetro “`required`”, o que significa que ela retorna -1 caso não haja máquinas livres. No entanto, uma nova chamada da função é feita nesse caso, porém, com “`required`” “`true`”, o que faz com que a execução do programa pare completamente caso não haja nenhuma máquina livre (`panic()`).

Ainda sobre a matriz de LEDs e na mesma função sobre a qual se disserta anteriormente, há uma chamada de “`ws2812b_program_init()`”, outra função no arquivo cabeçalho referente ao PIO. Essa função inicializa o GPIO usado (7), usa “`pio_sm_set_consecutive_pindirs()`” para setar a direção PIO no pino como de saída (passando os valores da instância do PIO e da máquina de estados como argumentos), atribui configurações padrão a uma variável com tipo de estrutura adequada com “`ws2812b_program_get_default_config()`”. Feito isso, o programa usa as três funções a seguir: “`sm_config_set_sideset_pins()`”, “`sm_config_set_out_shift()`”, “`sm_config_set_fifo_join()`” que, respectivamente, *seta* pinos de *sideset* da máquina de estados (salvando na estrutura de configuração), define os parâmetros de transferência da máquina de estados como tendo um threshold de 8 bits com *shift* para a direita e *autopull* e, a última, configura a máquina de estados para usar exclusivamente o “TX”. Por fim, é definido o valor do prescaler para que haja dez ciclos por transmissão e, são chamadas “`pio_sm_init()`” e “`pio_sm_set_enabled()`” para iniciar a máquina de estados PIO no hardware PIO iniciando a memória de programa no *offset* do programa PIO e, a função “`pio_sm_set_enabled()`”, responsável por finalmente habilitar a máquina de estados PIO.

Na sequência do que foi exposto, são inicializados os botões A e B, por meio de três funções: “`gpio_init()`” que inicializa o pino GPIO; “`gpio_set_dir()`” que *seta* o a direção do GPIO como unicamente de entrada e; “`gpio_pull_up()`”, que configura resistores de *pull-up* internos nos GPIOs em que cada botão está ligado. Após isso, tem-se, ainda, a inicialização do joystick, que começa com uma chamada de “`setup_joystick()`”. Dentro de tal função, a primeira instrução é uma chamada de “`adc_init()`”, que inicializa o hardware de ADC. Em seguida, é chamada duas vezes a função “`adc_gpio_init()`” para inicializar os GPIOs 26 e 27 para uso como ADC, respectivamente responsáveis por captar os eixos Y e X do joystick. Por fim, é feita a inicialização do pino de leitura do botão do joystick do mesmo modo como foi feita a inicialização dos botões.

Por fim, há a inicialização do Display, que se dá por meio da função “i2cInitDisplay()” que inicia o hardware I2C usando “i2c_init()”, define as funções de SDA e SCL aos pinos devidos (14 e 15) usando “gpio_set_function()”, configura resistores de *pull-up* internos nos GPIOs com “gpio_pull_up()” e, chama a função “ssd1306_init()” que possui o processo completo de inicialização do display. Essa função usa uma lista de comandos baseada nos valores definidos no arquivo “ssd1306_i2c.h”, valores de comandos encontrados no *datasheet* do ssd1306 e, os envia para o periférico para inicializá-lo definitivamente. Ademais, prepara-se a área de renderização com uma estrutura do tipo “render_area” e é calculada a parcela do *buffer* a ser destinada à área de renderização.

Configurações dos Registros:

As funções de configuração de registro e sua descrição se apresentam a seguir (com base no estudo do código do *SDK* e na referência 4):

- “stdio_init_all()” - Chama “stdio_usb_init()” a qual recorre a “tusb_init()”. A partir de então, é encargo do TinyUSB inicializar a interface USB. Para tanto, ocorre a alteração de registradores relacionados ao USB da placa.
- “gpio_set_function()” - Seleciona a função de um pino GPIO e, para isso, altera o registrador de controle relacionado a tal GPIO (GPIOx_CTRL, para um GPIO x).
- “pwm_init()” - Altera registradores relacionados ao controle de um slice PWM: o de *wrap*, o de divisor de clock, o de controle e o de comparação para ajustar o *duty cycle* (CHx_TOP, CHx_DIV e CHx_CSR, CHx_CC para um canal x).
- “pwm_set_gpio_level()” - *Seta* o valor do *duty cycle* e, portanto, altera o valor de CHx_CC, como a função anterior.
- “ws2812b_program_init()” - Chama várias funções para inicializar o programa de PIO usado para comunicação com a matriz de LEDs e que estão relacionadas a PIO (principalmente à *state machine* que possui registradores como SMx_PINCTRL, SMx_INSTR e outros) e GPIO e, portanto, modifica registradores relacionados ao PIO e ao GPIO (este último, em partes, como na próxima função descrita). Esta função é usada em “npInit()”, que inicializa a matriz.
- “pio_gpio_init()” e “gpio_init()” - Usadas respectivamente para inicializar um pino GPIO para uso com bloco PIO e, a segunda, para inicializar um GPIO para habilitá-lo para *I/O*. Ambas alteram o registrador de controle do GPIO referente, GPIOx_CTRL.
- “adc_init()” - Inicializa o hardware ADC e, por consequência, modifica registradores relacionados ao ADC.
- “adc_gpio_init()” - Inicializa um pino GPIO para funcionar com ADC, logo, altera registradores relacionados a ADC e ao GPIO.
- “gpio_pull_up()” - Determina o uso de resistores de *pull-up* internos em um GPIO e que, por conseguinte, altera registradores relacionados a isso.
- “i2c_init()” - Inicializa o hardware I2C e, por conseguinte.
- “gpio_set_function()” - *Seta* a função de um GPIO, logo, modifica o registrador de seleção de função do GPIO, ou seja, modifica o GPIOx_CTRL na parte referente à função, o campo FUNCSEL.

Estrutura e Formato dos Dados:

Quanto ao arquivo principal, “TLMGame.c”, tem-se poucos dados específicos, sendo todos apenas variáveis dos tipos uint, uint8_t, uint16_t, uint32_t, bool e, o mais específico entre eles, variáveis do tipo vetor de char ou, em outras palavras, strings. Ademais, faz-se uso, ainda nesse arquivo, de dados do tipo “absolute_time_t” (uint64_t) e “repeating_timer”, este

último uma estrutura do *SDK* da Pico que possui um `int64_t` (define de quanto em quanto tempo o timer gera *callback*), um ponteiro de void (armazena informações do usuário a serem passadas para a função de *callback*), um `int32_t` (“alarm_id_t”, armazena o id do alarme), uma variável do tipo “repeating_timer_callback_t”, que armazena o endereço para a função de *callback* e, uma variável do tipo ponteiro de “alarm_pool_t”. Este último tipo, por sua vez, é ponteiro para uma variável que é outra estrutura, composta por variáveis do tipo `uint8_t`, `int16_t`, `bool`, ponteiro de “alarm_pool_timer_t”, ponteiro de “spin_lock_t” e ponteiro de “alarm_pool_entry_t”. Mais a fundo, é possível notar ainda que essa última é também uma estrutura, no entanto, não cabe aqui falar mais a fundo a respeito do *SDK* da Pico.

Analisando, então, os demais arquivos, tem-se:

- `neopixel.h` - Define-se uma estrutura com três `uint8_t`, para armazenar os valores RGB de um LED da matriz (`npLED_t`);
- `ssd1306_i2c.h` - Definem-se duas estruturas, “render_area” e “ssd1306_t”. A primeira armazena quatro `uint8_t`, uma com a coluna de início, uma com a última coluna, uma com a página inicial e uma com a página final do display. Além disso, contém um inteiro indicando o tamanho do *buffer*. Já a segunda estrutura contém dados em `uint8_t` de largura, altura, páginas e endereço do display, além de um ponteiro de “i2c_inst_t” (estrutura relacionada à porta I2C do display), um `bool` “external_vcc”, um ponteiro de `uint8_t` “*ram_buffer”, um `size_t` “bufsize” (tamanho do *buffer*) e, um vetor de `uint8_t` “port_buffer” representando o *port* do *buffer*.

Organização da Memória:

Inicialmente, cabe ressaltar que, como padrão, o programa está sendo salvo na memória *flash* da PI PICO W. Nesse contexto, vale ressaltar também que não houve alocação dinâmica de memória no projeto, sendo todas as variáveis definidas diretamente. Logo, o programa principal depende da pilha da memória. Ainda assim, é importante pontuar que há também variáveis estáticas, como “static absolute_time_t click_time_A” ou a “static int count” que assumiam papel em funções de *callback* do programa e precisavam ter seus valores mantidos entre passagens pela função. Portanto, pode-se afirmar, também, que essa parte dos dados não se encontra na pilha e, por conseguinte, há uso de outra região da memória RAM. Finalmente, cabe destacar que, durante o desenvolvimento do programa, buscou-se usar o mínimo de memória possível sem que isso afetasse o funcionamento esperado do programa e sem afetar seu desempenho

Protocolo de comunicação:

Existe no projeto presente o uso do protocolo I2C para comunicação com o Display OLED (`ssd1306`). Tal protocolo está usando os GPIOs 14 e 15, o primeiro como SDA e o segundo como SCL. Esse protocolo trabalha de maneira síncrona, half-duplex serial e com apenas duas linhas, o SDA e o SCL, tendo, portanto, SCL como linha de clock e SDA como linha única para dados e controle de ambos os comunicadores. Ademais, para informações de inicialização, deve-se analisar a seção de Inicialização anterior. Finalmente, embora não seja um protocolo de comunicação propriamente dito, há a comunicação com a matriz de LEDs, feita por PIO. A máquina PIO em questão está configurada para possuir dez ciclos de clock por transmissão, as quais são de oito bits com *shift* à direita.

Formato do Pacote de Dados:

Os pacotes existentes no projeto aqui apresentados são pacotes para comunicação I2C e para a comunicação com a matriz de LEDs usando PIO. Nesse último caso, os pacotes têm oito bits e são formados por meio da função “npWrite()” que envia cada `uint8_t` referente a cada cor de cada LED de

cada vez, usando chamadas de “pio_sm_put_blocking”. Já os pacotes I2C, são enviados usando chamadas “i2c_write_blocking()”, responsáveis por criar e enviar pacotes partindo de um endereço de 7 bits do periférico, um ponteiro para o dado a ser enviado, as portas I2C pelas quais deve sair e o comprimento em bytes do dado a ser transmitido. Cabe ressaltar que na comunicação I2C o bit enviado logo após os sete de endereço é definidor do tipo de operação (leitura ou escrita), logo, por ser uma função de escrita, o pacote já é feito com o valor determinando escrita. Além disso, vale ressaltar que não há, no entanto, pacotes formais como os de redes sendo feitos no projeto, justamente por não haver o uso de redes de computadores (não há pacotes TCP/IP por exemplo).

Execução do projeto:

Metodologia:

Inicialmente, foi pensado o escopo do projeto e seus requisitos. Partindo do que há disponível na Bitdoglab, buscou-se pensar em algum jogo educacional ou que exercitasse o cérebro do jogador de algum modo. Durante esse processo, foram pensadas duas ideias que se sobressaíram, um jogo que se assemelhasse a testes de QI com conjuntos de figuras que seguissem uma lógica e, entre as quais houvesse uma lacuna para que aquele que o executa completasse a sequência. Já a segunda ideia, conforme se pode imaginar já conhecendo o projeto até este ponto, seria um jogo de matemática simples, mas que implementasse funcionalidades diferentes daqueles já existentes. Esse último jogo deveria conter contas que fossem tão mais difíceis quanto mais o usuário acertasse.

No contexto apresentado, tendo definidas as ideias a serem levadas a cabo, iniciou-se um processo de pesquisas que, mais futuramente, gerou o repositório no Github com projetos correlatos (referência de número 12 deste documento). Nesse processo, apesar de toda a busca, conforme já explicado, não foi possível encontrar projeto algum que se assemelhasse o bastante à ideia do jogo de lógica para serem considerados como correlatos. Ainda assim, quanto ao jogo de matemática, provavelmente por conta de sua menor especificidade, foi possível encontrar três projetos que melhor se assemelhavam à ideia proposta. Isso, de forma que, conforme explicado anteriormente, embora se assemelhassem ao presente projeto, nenhum deles estava de fato muito próximo. Isso porque, um usava apenas o terminal por USB para trabalhar entradas e saídas (com uma PI PICO) e pecava na inexistência de lógica de dificuldade progressiva e, outro, apesar de apresentar já um display, carecia também na lógica de progressão (embora tenha um vetor com nomes das dificuldades, ele não é usado no código apresentado). Já o terceiro, trata-se de um jogo um pouco mais elaborado, mas feito para uso no próprio computador com a geração de arquivo executável. Ainda assim, nenhum deles é feito em C e nem apresentavam o conjunto de *features* desejadas para o projeto a ser desenvolvido.

Em sequência, foram estabelecidos os requisitos, sobre os quais já se tratou neste documento e, foram, então, definidas as necessidades de hardware para o sistema. Para o jogo de lógica, o equipamento que se mostrou mais capaz de satisfazer as condições desejadas para interface visual foi a matriz de LEDs. Isso porque, ela seria capaz de apresentar quatro quadros de quatro LEDs cada, de forma que cada quadro não seria confundível com os demais e, haveria a possibilidade de atribuir cores diferentes a cada LED, permitindo a aleatoriedade na criação de fases. Além disso, a matriz se mostrou muito útil, também, no jogo de matemática, pois poderia ser usada para apresentar as contas, algarismo por algarismo (e operador), de maneira a treinar também a atenção do jogador. Ademais, o segundo componente que se apresentou na lista utilizada foi o display OLED, que apresentaria todas as mensagens necessárias para cumprir com os requisitos e, durante os jogos, seria capaz de apresentar o número da fase em que o jogador está e, no caso do jogo de matemática, até mesmo participar ativamente do processo de resposta do jogador (algarismos da resposta são apresentados no display). Além desses, decidiu-se por usar, ainda, os botões e o joystick, tanto para lógicas de escolhas durante as mensagens pré-jogo (A ou B, para escolher entre possibilidades) e, durante os jogos, para

uso do jogador como mecanismos para alterações em suas respostas: joystick, no jogo de lógica, moveria o cursor e confirmaria a escolha do quadro manipulável e, no jogo de matemática, alteraria o valor de um algarismo da resposta; A mudaria cor do cursor no jogo de lógica e permitiria o jogador a navegar de um algarismo mais significativo para um menos no jogo de matemática e; B permitiria que o jogador *setasse* a cor de um LED manipulável no jogo de lógica com a mesma cor do cursor e, no jogo de matemática, possibilitaria a navegação por algarismos mais significativos. Finalmente, os demais periféricos de hardware que se optou por usar foram os buzzers, responsáveis por emitir todos os sons do jogo.

Por fim, foram analisadas questões de software. Para tanto, em paralelo com a tentativa de estabelecer especificações a esse respeito, foram feitas análises dos códigos presentes nos exemplos da Bitdoglab em C (referência 13 a 16). Com isso, algumas especificações quanto as funções foram determinadas basicamente por funções já presentes nesses códigos. No entanto, havia ainda especificações a serem feitas a respeito das demais funções, que tratariam, principalmente, da lógica do programa no geral e dos jogos individualmente. Nesse ponto, especificaram-se funções como a de verificação de vitória, que usaria as respostas corretas de cada fase para verificar se a resposta do usuário é igual, a de derrota, que reiniciaria o programa após o jogador cometer um erro e, a de mensagens iniciais, que se incumbiria de apresentar no display as mensagens iniciais do programa, que viessem antes dos jogos e possuísem ainda escolhas para o jogador fazer. Dever-se-ia, ainda, haver a possibilidade de pular tutoriais, jogar sem som e escolher entre jogos. Para isso, deveriam haver variáveis que armazenassem as escolhas do usuário e fossem usadas para garantir que as vontades dele fossem satisfeitas.

Definidas as questões apresentadas, o projeto foi implementado utilizando-se a IDE VisualStudio Code, com extensões do Cmake e da Raspberry Pi Pico para criação e compilação correta do projeto. Durante sua implementação, foram feitos testes de unidade sobre a maioria das funções individualmente ou, em alguns casos, foram usadas outras funções que conviessem para o teste (principalmente funções de infraestrutura, que permitissem comunicação com periféricos, por exemplo). Ainda assim, mesmo durante tais testes, por vezes, foram percebidos erros, os quais tinham de ser sanados. Para tanto, foi utilizada a interface USB, com uso do Putty para depuração do programa utilizando chamadas da função “printf()” em posições estratégicas no código. Finalmente, ao final do projeto, como se explica profundamente adiante, foram feitos testes de validação para determinar se ele estava ou não funcionando de acordo com o desejado.

Testes de validação:

Feito e implementado todo o projeto, iniciou-se a fase de testes para validação de seu funcionamento. Essa fase foi iniciada com um teste básico de funcionalidades que iam além dos jogos, tais quais a possibilidade de jogar sem som, a possibilidade de pular tutoriais e a possibilidade de escolher entre jogos. Para tanto, os primeiros testes abrangeram todas essas possibilidades, de forma que, com o programa rodando, foram pressionados, durante as mensagens iniciais, os botões referentes às diversas escolhas de modo que fosse possível testar o funcionamento de cada uma delas. Ainda nesse meio, para garantir que as funcionalidades funcionavam corretamente entre jogos, a cada teste, o programa não era reiniciado com o botão de reset, mas sim com a derrota em um ou outro jogo.

Seguindo, após os testes referentes às escolhas possíveis para o usuário e, por consequência, referentes também às mensagens iniciais, foram feitos testes em cada um dos jogos e com intercalação entre eles. Isso se deu, de modo que, inicialmente, foram testadas as funcionalidades em uma única fase do jogo de lógica (fase 1), com o jogador movimentando o joystick e pressionando os botões para chegar a resposta correta e, em seguida, após a vitória, garantindo sua derrota. Isso se deu, também, para o jogo de matemática e, para garantir que não havia problemas com o reinício do programa após

uma derrota, foi testado o jogo de matemática após o de lógica e o de lógica após o de matemática. Além disso, vale ressaltar, ainda, que no jogo de lógica, para testar cada uma das lógicas possíveis (180 graus de rotação do primeiro para o segundo quadro de cada sequência, 90 graus em um dos sentidos e, mudanças de cores), foram feitos testes do jogo com alterações no código que garantissem a aparição de uma mesma lógica durante todas as fases do jogo, permitindo que fossem encontrados problemas em cada lógica, como o de o primeiro quadro de cima possuir pares de LEDs diagonalmente opostos de mesma cor na lógica de rotação de 90 graus, que incapacitaria o usuário de saber qual o sentido do movimento. Já no jogo de matemática, foram feitos testes semelhantes, considerando cada operador, mas, mais que isso, como haveria a progressão de dificuldade a cada conta acertada (vitória em fase), foi manipulado o código para que fosse possível testar valores mais altos (fases mais avançadas).

Finalmente, foram feitos testes com pessoas alheias ao programa, de modo que fosse possível ver o uso despretenso do projeto e, por consequência, que fosse possível certificar seu bom funcionamento e a necessidade de alterações ou correções como usuário externo. Ademais, apenas para pontuar, algumas funcionalidades, como a de o usuário já começar cada fase do jogo de matemática com o algoritmo selecionado para manipulação ser o mais significativo que a conta da fase exigiria, foi sugerido por uma pessoa alheia que o testou e notou a carência de algo desse tipo e, a necessidade que incomodava de ter que navegar desde o algoritmo mais significativo até, possivelmente, o menos entre cinco para começar a escrever a resposta.

Discussão dos Resultados:

Analisando a fundo o funcionamento do projeto final, após vários testes e também uso despretenso, pode-se concluir que o projeto atende a todos os requisitos e, é bastante confiável e aplicável. Partindo-se do princípio que sua aplicabilidade é pautada na posse de uma Bitdoglab, pode-se afirmar que é plenamente possível que qualquer pessoa, na posse de tal sistema, o use para executar o projeto apresentado. Ainda assim, caso não se tenha uma Bitdoglab em mãos, o sistema pode, ainda, ser reproduzido partindo-se da lista de componentes disposta neste documento. Ainda nesse sentido, quanto à aplicabilidade no sentido de ser possível aplicar de fato o projeto no cotidiano das pessoas, afirma-se que é um projeto aplicável, no sentido que exerce uma função muito boa, auxiliar no desenvolvimento e amadurecimento dos raciocínios lógico e matemático dos usuários e, por se tratar de um jogo, poderia se disseminado entre o público mais jovem. Além disso, quanto à confiabilidade do projeto, pode-se afirmar que, com exceção de problemas que possam ocorrer com o hardware do sistema a mais longo prazo, o sistema é muito protegido de qualquer tipo de erro e é extremamente capaz em exercer as funções que lhe foram atribuídas de maneira muito satisfatória.

Apresentação do Projeto (Vídeo):

A seguir, exibe-se o *link* para um vídeo demonstrativo do projeto funcionando. O vídeo se exibe aqui em forma de um *link* para o *Youtube*:

<https://youtu.be/8H5eaY6LUW8>

Referências:

- 1 - von Davier, M., Kennedy, A., Reynolds, K., Fishbein, B., Khorramdel, L., Aldrich, C., Bookbinder, A., Bezirhan, U., & Yin, L. (2024). TIMSS 2023 International Results in Mathematics and Science. Boston College, TIMSS & PIRLS International Study Center. Disponível em: <<https://doi.org/10.6017/lse.tpisc.timss.rs6460>>
 - 2 - Datasheet WS2812B. Disponível em: <<https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf>>.
 - 3 - Slide sobre Clocks e temporizadores do curso Embarcotech. Disponível em: <https://ead.hardware.org.br/course/format/tiles/mod_view.php?cmid=303>.
 - 4 - Datasheet Raspberry Pi RP2040. Disponível em: <<https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>>.
 - 5 - Datasheet Raspberry Pi Pico W. Disponível em: <<https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>>.
 - 6 - Datasheet MLT-8530. Disponível em: <https://www.lcsc.com/datasheet/lcsc_datasheet_2410010301_Jiangsu-Huaneng-Elec-MLT-8530_C94599.pdf>.
 - 7 - Datasheet SSD1306. Disponível em: <<https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>>.
 - 8 - Tabela “Custo de material para BITDOG picoLab 26-07-23”. Disponível em: <<https://docs.google.com/spreadsheets/u/1/d/10G9U2lKZ8DwIemRyy8-OiIrZH5e2oOeCSGOkK32-5-8/edit?gid=1509692557#gid=1509692557>>.
 - 9 - Banco de informações da Bitdoglab (português). Disponível em: <<https://docs.google.com/document/d/13-68OqiU7lSE8U2KPRUXT2lSeBl3WPhXjGDFH52eWIU/e dit?tab=t.0>>
 - 10 - Página github Project TinyUSB. Disponível em: <<https://github.com/hathach/tinyusb/tree/master>>.
 - 11 - Repositório do projeto desenvolvido (TLMGame). Disponível em: <<https://github.com/viniciusemantovani/TLMGame>>.
 - 12 - Repositório de projetos correlatos. Disponível em: <https://github.com/viniciusemantovani/Correlatos_Embarcotech.git>.
- Códigos usados no desenvolvimento das Bibliotecas de comunicação com periféricos:
- 13 - Código exemplo em C para uso da matriz de LEDs da Bitdoglab. Disponível em: <https://github.com/BitDogLab/BitDogLab-C/tree/main/neopixel_pio>.
 - 14 - Código exemplo em C para uso do display OLED da Bitdoglab. Disponível em: <https://github.com/BitDogLab/BitDogLab-C/blob/main/display_oled>.
 - 15 - Código exemplo em C para uso do Joystick da Bitdoglab. Disponível em: <https://github.com/BitDogLab/BitDogLab-C/blob/main/Joystick_led>.
 - 16 - Código exemplo em C para uso dos Buzzers da Bitdoglab. Disponível em: <https://github.com/BitDogLab/BitDogLab-C/tree/main/buzzer_pwm>.